

Entregable 2

Taller de Aprendizaje Automático

Estéfano Bargas

Junio 2024

1. Demanda de bicicletas

1.1. Parámetros de los modelos

Para los modelos utilizando la capa simple, la cantidad de parametros es de 7937, de las cuales 7872 corresponden a la capa recurrente y 65 a las conexiones de las 64 unidades mas el termino de bias a la neurona de salida. Los parámetros de la capa recurrente corresponden a los pesos de entrada (dependiente de la cantidad de features, 58 en nuestro caso luego de preprocesar), los pesos de recurrencia (la realimentacion de las unidades) y los biases correspondientes. Los modelos que utilizan LSTM tienen una capa recurrente con 31488 parámetros, mucho mayor a la capa simple ya que cada unidad LSTM se trata de un conjunto de compuertas que controlan el flujo de los datos de manera mas compleja, con el objetivo de evitar el problema de desvanecimiento de gradientes.

No existe diferencia de cantidad de parámetros entre los modelos *Seq-to-Vec* y *Seq-to-Sec* ya que la diferencia de estos ultimos es que producen un valor de salida por cada unidad de tiempo, en vez de por cada secuencia. Justamente una de las ventaja de las redes recurrentes es aprovechar la dependencia temporal de los datos para minimizar el tamaño del modelo (si se *desenrolla* la capa recurrente se obtendría la red densa equivalente, multiplicando la cantidad de parámetros por la dimensión temporal).

1.2. Rendimiento de los modelos

Los modelos creados consisten en una capa recurrente de 64 unidades, seguida por una capa densa de una neurona con activacion lineal y en el caso de los modelos Seq-to-Sec esta fue envuelta en el wrapper *TimeDistributed*, esto hace que se aplique la capa densa para la salida de la capa recurrente de cada unidad de tiempo, aprovechando la secuencia entera. En la [Tabla 2](#) se presentan los rendimientos para todos los modelos entrenados en el taller.

MAE, RMSLE	Naive	Simple Seq-to-Vec	Simple Seq-to-Sec	LSTM Seq-to-Vec	LSTM Seq-to-Sec
Entrenamiento	65, 0.6470	142, 1.4330	142, 1.4385	145, 1.4413	139, 1.4223
Validación	65, 0.6470	142, 1.3986	129, 1.4086	146, 1.4113	126, 1.3947

Cuadro 1: Rendimiento de los modelos del taller 8.

Para comenzar, los modelos Seq-to-Vec no estaban entrenando por defecto (la metrica de perdida se mantenía constante), siguiendo el modelo sugerido por el taller. Probando activar el atributo *stateful* de la capa recursiva para aprovechar los pesos del epoch anterior los modelos comenzaron a entrenarse, sin embargo tanto Simple como LSTM terminan con un rendimiento peor que la técnica Naive. Ocurrió el mismo problema para los modelos Seq-to-Vec, sin embargo la solución no fue activar *stateful* sino que agregar una capa *BatchNormalization* entre la recurrente y la capa densa final. Esto sugiere que el modelo estaba sufriendo del problema de desvanecimiento de gradientes ya que la metrica de salida se mantenía constante pero finita y el modelo no entrenaba. Sabiendo esto uno esperaría un mucho mejor rendimiento en los modelos LSTM (pues esta capa fue diseñada específicamente para atacar este problema), sin embargo no es el caso. Para descartar un posible problema con el optimizer, se probaron diferentes ordenes de magnitud del learning rate y utilizar *Nadam* en vez de *Adam*, sin éxito.

Como el problema parece sistematico en todos los modelos, se hipotetizo que puede existir un error en el procesamiento de los datos, específicamente al crear el dataset en serie temporal (pues el preprocesamiento es el entregado por los docentes y modificado para eliminar las columnas *registered* y *casual*). Se probó distintas maneras de crear los datos, tanto para obtener secuencias o vectores como salida, obteniendo los mismos resultados cada vez.

Observando las predicciones para un mes del conjunto de entrenamiento en la [Figura 1](#) para el modelo Simple Seq-to-Sec se observa que el modelo está optimizando a predecir un promedio de alquileres, en vez de ajustarse a la serie. Luego de varias pruebas se llegó a que la causante de esto era el preprocesamiento brindado por los docentes, ya que se estaba dejando pasar features sin normalizar y provocando que estas tengan mayor incidencia que todo el resto. Eliminando estas se comenzó a obtener mejores predicciones, presentadas en la [Figura 2](#). La razón por la cual los modelos Seq-to-Sec tienen mejor rendimiento puede deberse al hecho de que se utiliza un stride de 24 para los datos seq-to-seq, por lo cual hay menos muestras por época. Sin embargo al arreglar este detalle aun así los modelos seq-to-seq no llegan al rendimiento de uno set-to-vec.

MAE, RMSLE	Naive	Simple Seq-to-Vec	Simple Seq-to-Seq	LSTM Seq-to-Vec	LSTM Seq-to-Seq
Entrenamiento	65, 0.6470	58, 0.3814	73, 0.4714	24, 0.2728	65, 0.4025
Validación	65, 0.6470	62, 0.4036	77, 0.5306	39, 0.3839	47, 0.4891

Cuadro 2: Rendimiento de los modelos del taller 8 posterior a resolver el preprocesamiento de los datos.

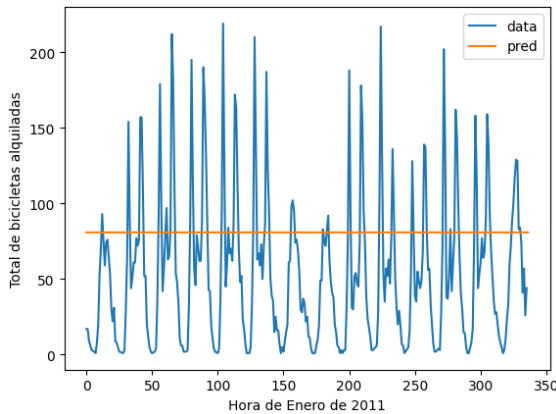


Figura 1: Predicciones del modelo Simple Seq-to-Seq utilizando features no normalizadas.

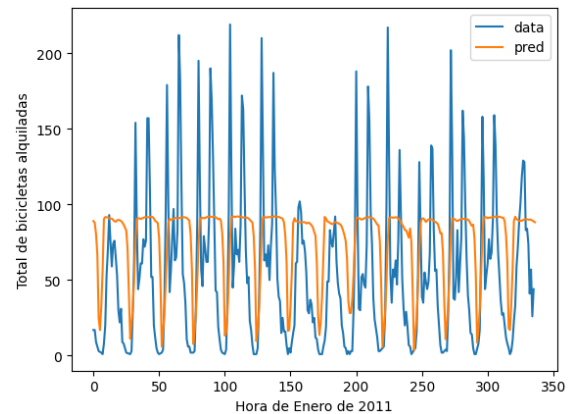


Figura 2: Predicciones del modelo Simple Seq-to-Seq dropping las features no normalizadas.

1.3. Mejora de los modelos

Dado lo discutido en la sección anterior se toma como base el modelo LSTM Seq-to-Seq y se trata de aumentar su complejidad y aplicar alguna técnica de regularización de ser necesario. Se agregó una capa recurrente mas con 64 unidades, esta vez Seq-to-Vec. El rendimiento converge a un valor alto de error y obteniendo nuevamente el mismo problema inicial explicado en la sección anterior.

Luego se probó agregar una capa densa de 64 unidades al modelo LSTM Seq-to-Vec, sin cambios significativos en el rendimiento mas que aumentar el sobreajuste del modelo. Se probó además agregar una capa *Dropout* posterior a la capa densa extra, pero solamente empeoró el rendimiento en validación. El mejor modelo entonces se trata del sencillo LSTM Seq-to-Seq, con un error promedio de 39 bicicletas alquiladas, correspondiendo a un MAPE de 17%. En la figura [Figura 3](#) se muestran predicciones de este modelo para datos de validación.

2. Critica de películas

2.1. TextVectorization

Esta capa codifica una cadena de texto en un vector de enteros o flotantes, dependiendo de la técnica que se elija. Por defecto convierte cada palabra en un entero, el cual es elegido a partir de un vocabulario ingresado por el usuario o entrenado de un arreglo de cadenas de texto. Utilizando el vocabulario generado a partir del conjunto de entrenamiento de este taller, se obtiene:

```
> text_vec_layer('Hello world')
<tf.Tensor: shape=(2,), dtype=int64, numpy=array([5655, 187])>
```

Fragmento 1: Ejemplo de la capa TextVectorization

La palabra 'Hello' se codifica a 5655, y 'world' a 187.

2.2. Largo de secuencia

Una red recurrente es alimentada por una secuencia de datos en serie, un dato a la vez. Por cada dato se obtiene una salida en cada neurona de la red, y luego estas son retroalimentadas con su propio estado (de aquí su denominación *recurrente*). En nuestro caso se toma en cuenta la ultima salida de cada neurona (a este tipo de red se le llama *Seq-to-Vec*). Bajo esta definición entonces la arquitectura de la red no depende del largo de cada secuencia. Sin embargo es común homogeneizar el largo de cada secuencia (por ejemplo con *zero-padding*) debido a otras razones, como puede ser eficiencia en el manejo de memoria o debido a otras capas del modelo que si sean sensibles al tamaño.

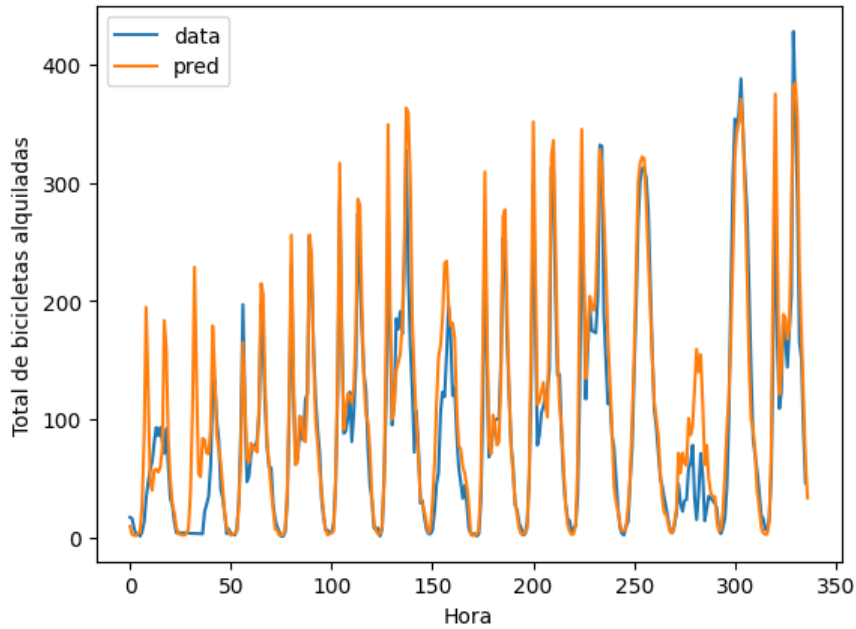


Figura 3: Predicciones del mejor modelo del taller 8 para los primeros 15 días del conjunto de validación.

2.3. Sobre embeddings

Una capa de embedding codifica datos discretos en vectores continuos. La ventaja de los embeddings pre-entrenados son:

1. Eliminar el costo computacional de entrenar la capa.
2. Tener una capa entrenada con un conjunto de datos mucho mas grande y diverso al que se suele tener disponible.

Aunque el segundo punto puede ser contraproducente, ya que si bien obtenemos una capa que generaliza mejor a distintos problemas, perdemos el contexto del problema actual. Si se trata de codificar cadenas de texto entonces puede ser perjudicial perder el contexto debido a la complejidad del lenguaje.

Entonces la ventaja clara de entrenar un embedding de cero es mantener el contexto, es decir, se da significado a cada dato a partir de las relaciones presentes en los datos del problema y en base al objetivo a cumplir, en vez de relaciones generales no específicas del contexto actual (o sea, un embedding mas general subajusta al conjunto de entrenamiento de un problema con corto alcance y vocabulario limitado o donde el contexto tenga un fuerte peso). Esto trae la desventaja del costo computacional de entrenamiento y de entrenar con un conjunto de datos mas pequeño. Una alternativa es utilizar un embedding pre-entrenado para un contexto similar al del problema a resolver, o reemplazar el embedding por un modelo de lenguaje natural pre-entrenado y realizar ajuste fino.

2.4. Embedding final

Se exploraron varias posibles mejoras para el modelo final (para el cual se utilizo un embedding entrenado desde cero debido a su mejor rendimiento):

- Cambiar el modo del embedding a *tf-idf*, pero el costo computacional termino siendo prohibitivamente alto.
- Duplicar el tamaño del vocabulario.
- Utilizar 2-gramas.
- Duplicar y dividir el tamaño de la salida del embedding.
- Utilizar una capa recurrente LSTM en vez de GRU.
- Duplicar y dividir la cantidad de neuronas de la capa recurrente.
- Utilizar AdamW para reemplazar la regularización l_2 por weight decay.
- Agregar una capa densa de tantas neuronas como salidas de la capa recurrente.

Como métrica de evaluación se tomo el área bajo la curva ROC. El mejor modelo encontrado utiliza:

- la capa TextVectorization por defecto (modo *int*), con un vocabulario de 20.000 tokens y zero-padding.
- Una capa embedding con Masking, regularización l_1 y tamaño de salida 128.

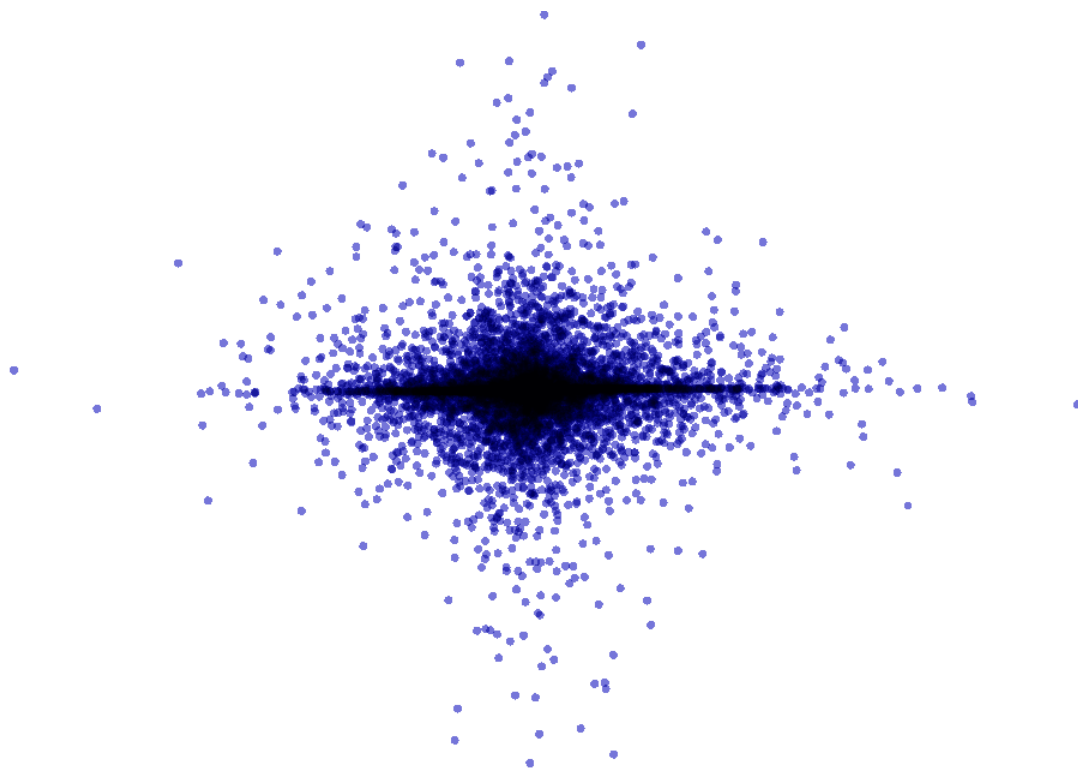


Figura 4: Las dos componentes principales del embedding final visualizado con *Embedding Projector*. Cada punto es el vector correspondiente a un token del vocabulario.

- Una capa GRU con misma cantidad de neuronas que de salidas del embedding.
- Una capa densa con activación sigmoide.
- Optimizador AdamW con learning rate por defecto y weight decay de 0,04, entrenando durante 10 epocas con early stopping.

Las medidas de este modelo se presentan en la [Tabla 3](#). El modelo no sobreajusta y el rendimiento en test es bueno, aunque el hecho de que el accuracy sea mejor en test mientras que AUC peor nos dice que el punto de operación no es óptimo y probablemente tengamos precisión y recall dispares. No se estudia mas a fondo el modelo en esta sección.

Conjunto	AUC	Accuracy
Entrenamiento	0.9729	92.2 %
Validación	0.9657	90.4 %
Test	0.8976	96.2 %

Cuadro 3: Rendimiento del modelo final del taller 9.

Se utilizo el *Embedding Projector* de Comet para visualizar el embedding entrenado. Se predijo los tokens del vocabulario utilizado y se muestran los dos componentes principales del resultado en la [Figura 4](#). A la izquierda se encuentran tokens asociados con reviews positivas (palabras como "exceptional", "flawless" y numeración como "710" y "810", siendo estos últimos posibles puntajes en base 10 otorgados por las reviews) y a la derecha negativos ("uninspired", "awful", "forgettable" y puntajes como "410" y "310"). Sobre el eje vertical se encuentran palabras neutras ("film", "australia", "all").

Para mejorar la visualización, en la [Figura 5](#) y [Figura 6](#) se visualiza la búsqueda de los tokens "110" y "810" correspondiendo a un buen y mal puntaje respectivamente, junto a los 10 vecinos mas cercanos de cada uno.

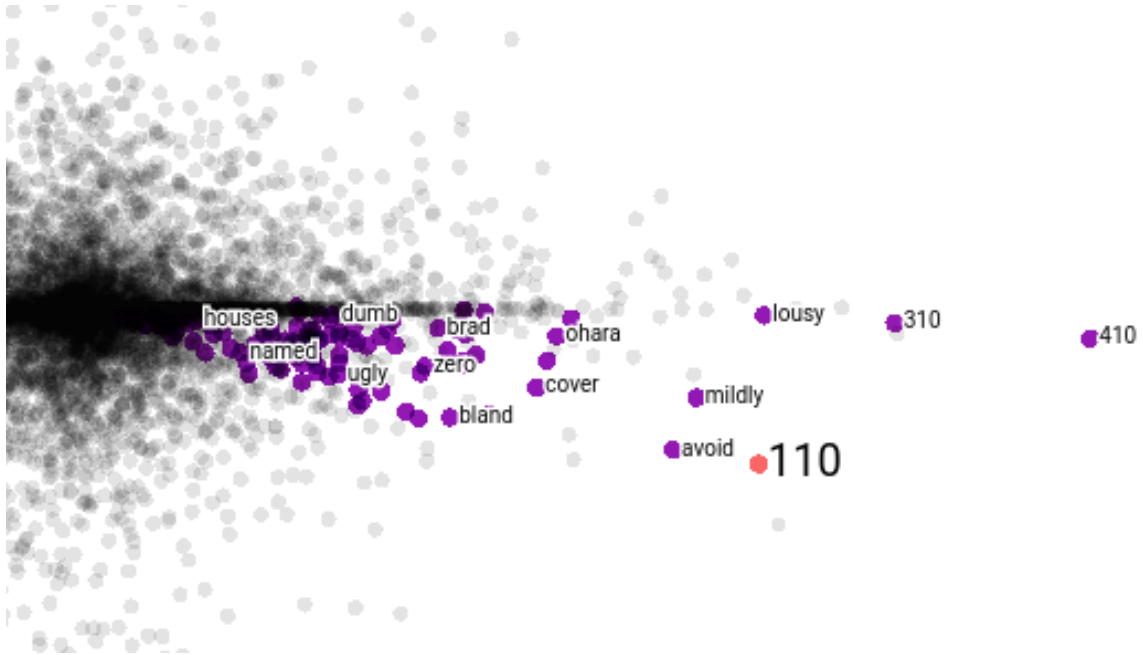


Figura 5: Token "110" correspondiente al peor puntaje y sus 10 vecinos mas cercanos.

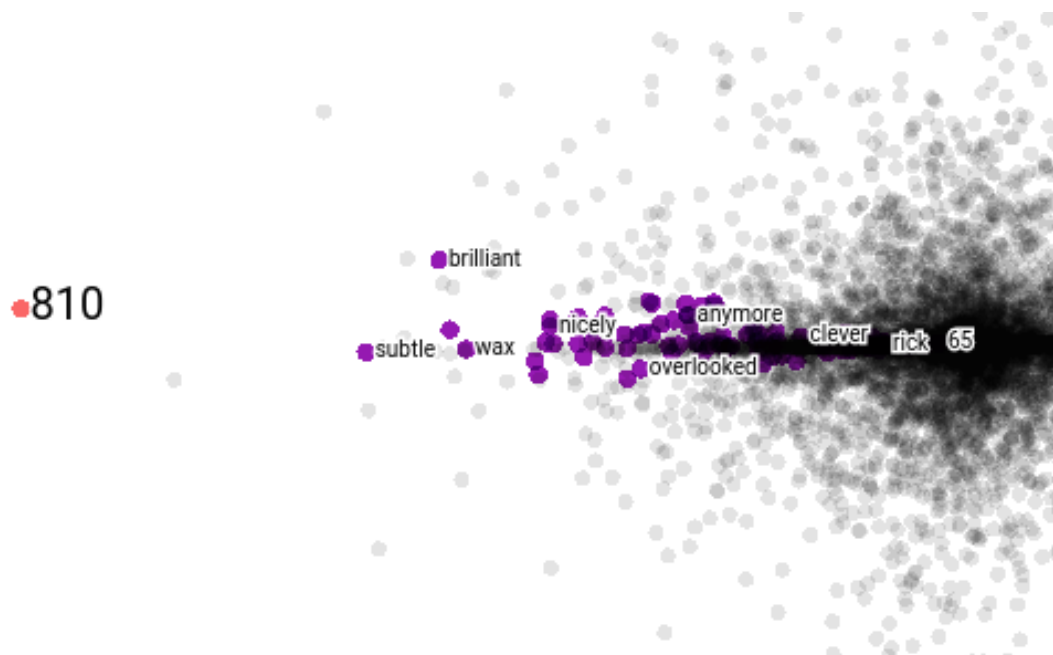


Figura 6: Token "810" correspondiente a un buen puntaje y sus 10 vecinos mas cercanos.